



Politechnika
Wrocławska



PROJEKT ZESPOŁOWY

Aplikacja webowa Trivia Game

Autorzy:

Gabriel MALANOWSKI 281081

Mateusz ŁUKASIEWICZ 281035

Bartosz PŁUCIENNIK 281439

Dominik KACZMAREK 281007

Ignacy SZADKOWSKI 281016

Prowadzący:

mgr inż. Weronika WĘGIER

WROCLAW, 13.03.2026R.

1 Wstęp

1.1 Cel projektu

Celem projektu jest opracowanie nowoczesnej aplikacji webowej umożliwiającej użytkownikowi rozgrywanie quizów wiedzy ogólnej. System ma pobierać pytania z Open Trivia Database API, prezentować je użytkownikowi, umożliwiać udzielanie odpowiedzi, obliczać wynik końcowy oraz prezentować zdobyte punkty.

1.2 Zakres projektu

Zakres projektu obejmuje przygotowanie kompletnej aplikacji webowej składającej się z frontendu, backendu oraz bazy danych. System będzie umożliwiał wybór kategorii i poziomu trudności, pobieranie pytań z zewnętrznego API, rozwiązywanie quizów, zapisywanie wyników oraz wyświetlanie rankingu użytkowników.

1.3 Krótki opis aplikacji

Aplikacja będzie działała w architekturze trójwarstwowej. Warstwa prezentacji zostanie przygotowana w technologii React, backend będzie udostępniał REST API, natomiast dane będą przechowywane w relacyjnej bazie danych obsługiwanej przez Prisma ORM. Komunikacja pomiędzy frontendem i backendem będzie odbywać się z wykorzystaniem formatu JSON.

2 Skład zespołu

2.1 Członkowie zespołu

- Gabriel Malanowski
- Mateusz Łukasiewicz
- Bartosz Płuciennik
- Dominik Kaczmarek
- Ignacy Szadkowski

2.2 Podział ról i odpowiedzialności

- Backend: Bartosz Płuciennik, Dominik Kaczmarek
 - implementacja REST API,
 - integracja z Open Trivia Database API,
 - implementacja logiki quizu,
 - obsługa odpowiedzi użytkownika,
 - zapis wyników w bazie danych.

- Frontend: Ignacy Szadkowski, Mateusz Łukasiewicz
 - przygotowanie interfejsu użytkownika,
 - ekran startowy,
 - wybór kategorii i poziomu trudności,
 - ekran pytań i odpowiedzi,
 - ekran końcowego wyniku.
- Project Manager i CI: Gabriel Malanowski
 - zarządzanie harmonogramem projektu,
 - koordynacja pracy zespołu,
 - konfiguracja repozytorium i workflow Git,
 - integracja frontend-backend,
 - wsparcie implementacji backendu,
 - przygotowanie dokumentacji technicznej.

3 Analiza problemu i przegląd istniejących rozwiązań

3.1 Opis problemu

Współczesne aplikacje quizowe często oferują ograniczony zakres funkcjonalności lub skupiają się wyłącznie na jednym typie użytkownika. Część dostępnych rozwiązań nie pozwala na wybór kategorii pytań, poziomu trudności lub nie oferuje przejrzystego interfejsu użytkownika. Problemem jest również brak prostych i nowoczesnych aplikacji webowych, które umożliwiłyby szybkie rozpoczęcie quizu bez konieczności instalacji dodatkowego oprogramowania.

Projektowana aplikacja ma rozwiązać ten problem poprzez udostępnienie intuicyjnego systemu quizowego dostępnego z poziomu przeglądarki internetowej. Użytkownik będzie mógł wybrać kategorię pytań oraz poziom trudności, a następnie rozwiązać quiz i zobaczyć końcowy wynik. Pytania będą pobierane z zewnętrznego API Open Trivia Database, co pozwoli na zapewnienie dużej liczby różnorodnych quizów.

3.2 Istniejące rozwiązania

Na rynku dostępnych jest wiele aplikacji quizowych i edukacyjnych. Do najpopularniejszych należą:

- Kahoot - platforma edukacyjna umożliwiająca tworzenie własnych quizów i rozgrywek wieloosobowych,
- Quizizz - aplikacja oferująca quizy edukacyjne oraz możliwość tworzenia własnych zestawów pytań,
- Trivia Crack - mobilna gra quizowa oparta na pytaniach z różnych kategorii,
- Sporcle - serwis internetowy umożliwiający rozwiązywanie quizów tematycznych,

Większość istniejących rozwiązań jest rozbudowana i zawiera wiele dodatkowych funkcji, takich jak konta użytkowników, tryby multiplayer, rankingi globalne czy system osiągnięć. W projektowanej aplikacji nacisk zostanie położony przede wszystkim na prostotę obsługi, przejrzysty interfejs oraz podstawowe funkcjonalności związane z rozwiązywaniem quizów.

3.3 Porównanie istniejących rozwiązań

Tabela 1: Porównanie istniejących rozwiązań quizowych

Aplikacja	Kategorie pytań	Poziomy trudności	Tryb rankingowy	Charakterystyka
Kahoot	Tak	Częściowo	Tak	Rozbudowana platforma edukacyjna
Quizizz	Tak	Tak	Tak	Narzędzie edukacyjne do quizów szkolnych
Trivia Crack	Tak	Nie	Tak	Mobilna gra quizowa
Sporcle	Tak	Nie	Nie	Serwis z quizami tematycznymi
Projektowana aplikacja	Tak	Tak	Tak	Prosta aplikacja webowa oparta o Open Trivia Database API

3.4 Wnioski z analizy rynku

Analiza istniejących rozwiązań pokazuje, że aplikacje quizowe cieszą się dużą popularnością, jednak wiele z nich jest nastawionych głównie na edukację lub rozrywkę w trybie wieloosobowym. Istnieje miejsce na prostą aplikację webową, która będzie skupiała się na szybkim i wygodnym rozwiązywaniu quizów wiedzy ogólnej.

Wnioskiem z analizy rynku jest potrzeba stworzenia aplikacji, która będzie:

- prosta w obsłudze,
- dostępna z poziomu przeglądarki internetowej,
- umożliwiała wybór kategorii i poziomu trudności,
- oferowała ranking i zapis wyników,
- posiadała nowoczesny i przejrzysty interfejs użytkownika.

4 Analiza wymagań użytkownika

4.1 Grupa docelowa

Grupą docelową aplikacji są przede wszystkim uczniowie, studenci oraz osoby zainteresowane quizami wiedzy ogólnej. Aplikacja może być wykorzystywana zarówno w celach rozrywkowych, jak i edukacyjnych.

Z systemu mogą korzystać użytkownicy, którzy:

- chcą sprawdzić swoją wiedzę z różnych dziedzin,
- chcą rywalizować z innymi użytkownikami,
- szukają prostego sposobu na naukę poprzez zabawę,
- oczekują szybkiego dostępu do quizów bez konieczności instalacji aplikacji.

4.2 Potrzeby użytkowników

Na podstawie analizy problemu można określić następujące potrzeby użytkowników:

- możliwość szybkiego rozpoczęcia quizu,
- wybór kategorii pytań,
- wybór poziomu trudności,
- intuicyjny i czytelny interfejs,
- możliwość sprawdzenia końcowego wyniku,
- zapis wyników i historii quizów,
- możliwość porównania swoich wyników z innymi użytkownikami.

Potrzeby te są zgodne z zakresem funkcjonalności planowanych w projekcie, takich jak pobieranie pytań z API, prezentacja pytań, obliczanie wyniku oraz ranking użytkowników.

4.3 Główne scenariusze użycia

1. Użytkownik otwiera aplikację i wybiera kategorię quizu.
2. Użytkownik wybiera poziom trudności pytań.
3. System pobiera pytania z Open Trivia Database API.
4. Użytkownik odpowiada na kolejne pytania.
5. System oblicza wynik końcowy.
6. Wynik zostaje zapisany w bazie danych.
7. Użytkownik może zobaczyć ranking oraz porównać swoje wyniki z innymi użytkownikami.

5 Założenia projektowe

5.1 Zakres MVP

Minimalna wersja produktu będzie obejmowała:

- ekran startowy aplikacji,
- wybór kategorii quizu,
- wybór poziomu trudności,
- pobranie pytań z Open Trivia Database API,
- wyświetlanie pytań i możliwych odpowiedzi,
- możliwość zaznaczania odpowiedzi,
- obliczanie wyniku końcowego,
- ekran końcowego wyniku.

Rozszerzenia, takie jak ranking użytkowników, historia quizów czy bardziej rozbudowany profil użytkownika, będą realizowane dodatkowo.

5.2 Główne funkcjonalności aplikacji

- wybór kategorii quizu,
- wybór poziomu trudności,
- pobieranie pytań z API,
- prezentacja pytań użytkownikowi,
- obsługa odpowiedzi,
- obliczanie wyniku,
- zapis wyników w bazie danych,
- ranking użytkowników.

5.3 Ograniczenia projektu

- projekt jest ograniczony czasowo do jednego semestru,
- funkcjonalność aplikacji zależy od dostępności zewnętrznego API,
- liczba funkcjonalności musi zostać dostosowana do możliwości czasowych zespołu,
- aplikacja będzie początkowo obsługiwała wyłącznie quizy wiedzy ogólnej.

5.4 Założenia technologiczne

- frontend: React.js,
- backend: Bun.js oraz Hono,
- ORM: Prisma,
- baza danych: PostgreSQL,
- komunikacja: REST API oraz JSON,
- repozytorium: GitHub.

6 Wymagania funkcjonalne

6.1 Rejestracja i logowanie użytkownika

System powinien umożliwiać użytkownikowi utworzenie konta oraz logowanie się do aplikacji. Rejestracja powinna wymagać podania podstawowych danych, takich jak nazwa użytkownika, adres e-mail oraz hasło.

Po poprawnym zalogowaniu użytkownik powinien uzyskać dostęp do historii swoich quizów, zapisanych wyników oraz rankingu. System powinien umożliwiać również wylogowanie się z aplikacji.

6.2 Rozpoczynanie quizu

Użytkownik powinien mieć możliwość rozpoczęcia nowego quizu z poziomu ekranu głównego aplikacji. Po wybraniu odpowiednich parametrów quizu system powinien pobrać zestaw pytań z Open Trivia Database API i przygotować quiz do rozpoczęcia.

6.3 Wybór kategorii i poziomu trudności

Przed rozpoczęciem quizu użytkownik powinien mieć możliwość wyboru:

- kategorii pytań,
- poziomu trudności.

System powinien przekazywać wybrane parametry do backendu, który pobierze odpowiednie pytania z API.

6.4 Rozwiązywanie quizu

Podczas rozwiązywania quizu użytkownik powinien widzieć treść pytania oraz listę możliwych odpowiedzi. System powinien umożliwiać zaznaczenie jednej odpowiedzi i przejście do kolejnego pytania.

Po zakończeniu quizu odpowiedzi użytkownika powinny zostać ocenione, a system powinien obliczyć końcowy wynik. Backend powinien odpowiadać za walidację odpowiedzi oraz zapis wyników do bazy danych.

6.5 Wyświetlanie wyników

Po zakończeniu quizu użytkownik powinien zobaczyć ekran wyników zawierający:

- liczbę poprawnych odpowiedzi,
- liczbę wszystkich pytań.

System powinien również umożliwiać powrót do ekranu głównego lub rozpoczęcie nowego quizu.

6.6 Ranking użytkowników

System powinien umożliwiać wyświetlanie rankingu użytkowników. Ranking może być tworzony na podstawie:

- liczby rozwiązanych quizów,
- procentu poprawnych odpowiedzi.

Ranking powinien być regularnie aktualizowany po zapisaniu nowych wyników quizów.

7 Wymagania niefunkcjonalne

7.1 Wydajność

Aplikacja powinna działać płynnie i umożliwiać szybkie ładowanie danych. Czas odpowiedzi backendu na większość zapytań nie powinien przekraczać 2 sekund.

Pobieranie pytań z Open Trivia Database API oraz wyświetlanie wyników powinno odbywać się bez zauważalnych opóźnień dla użytkownika.

7.2 Responsywność interfejsu

Interfejs użytkownika powinien być responsywny i dostosowany do różnych rozdzielczości ekranów. Aplikacja powinna poprawnie działać zarówno na komputerach, jak i na urządzeniach mobilnych oraz tabletach.

Układ strony powinien automatycznie dostosowywać się do wielkości ekranu użytkownika.

7.3 Bezpieczeństwo danych

System powinien przechowywać hasła użytkowników w postaci zaszyfrowanej. Dane logowania nie powinny być przechowywane w postaci jawnej.

Komunikacja pomiędzy frontendem i backendem powinna odbywać się z wykorzystaniem bezpiecznego protokołu HTTPS. Dostęp do danych użytkownika powinien być możliwy wyłącznie po zalogowaniu.

7.4 Dostępność aplikacji

Aplikacja powinna być dostępna przez całą dobę z wyjątkiem planowanych przerw serwisowych. System powinien być dostępny z poziomu popularnych przeglądarek internetowych, takich jak Google Chrome, Mozilla Firefox, Microsoft Edge oraz Safari.

7.5 Skalowalność

Architektura systemu powinna umożliwiać dalszy rozwój aplikacji w przyszłości. W kolejnych etapach projektu możliwe powinno być dodanie nowych funkcjonalności, takich jak:

- profile użytkowników,
- osiągnięcia i odznaki,
- quizy multiplayer,
- dodatkowe rankingi,
- obsługa wielu języków.

7.6 Niezawodność

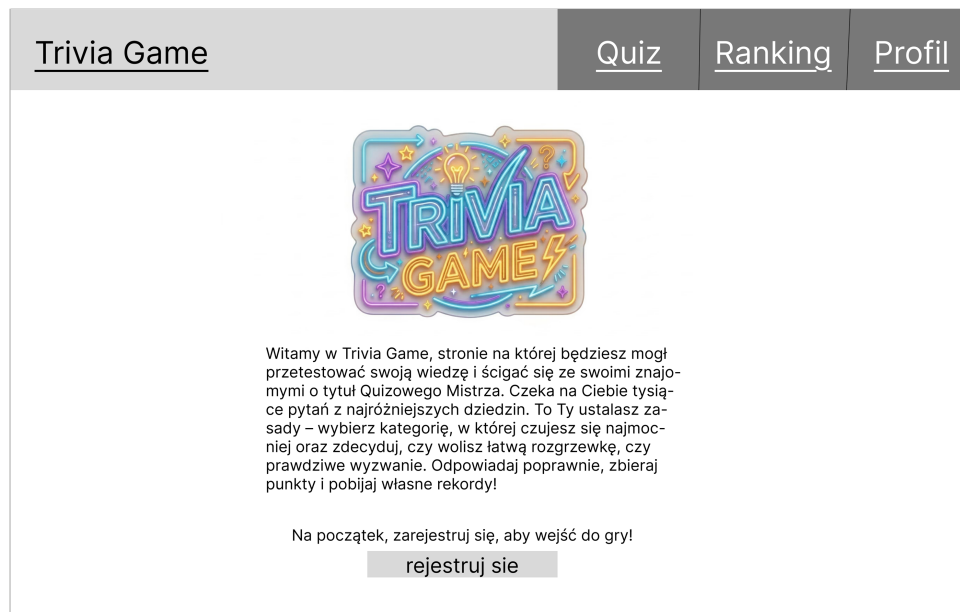
System powinien poprawnie obsługiwać błędy związane z niedostępnością zewnętrznego API, problemami z połączeniem internetowym lub błędnymi danymi wejściowymi.

W przypadku wystąpienia błędu użytkownik powinien otrzymać czytelny komunikat informujący o problemie oraz możliwość ponowienia operacji.

8 Projekt UI/UX

8.1 Ekran startowy

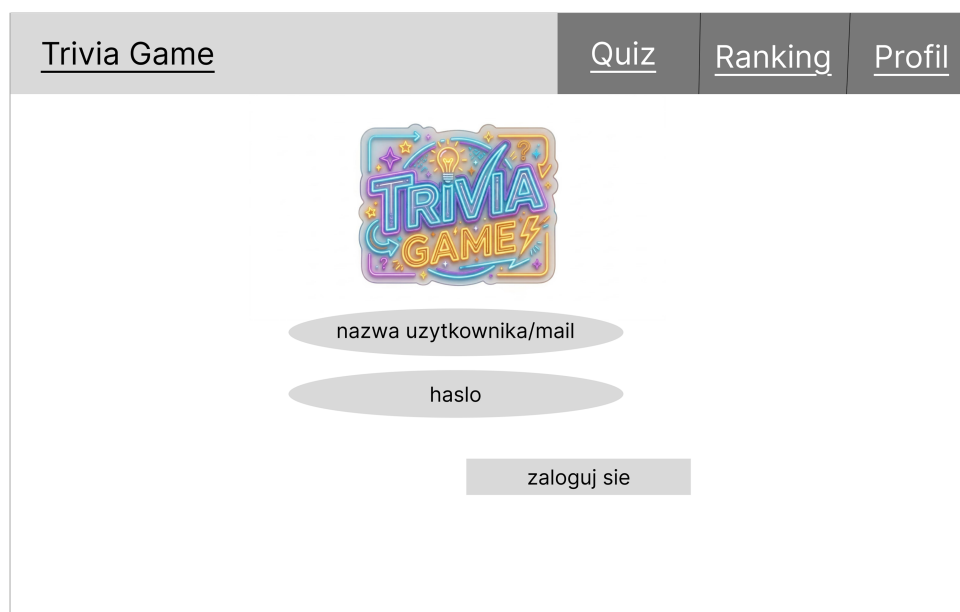
0



Rysunek 1: Makieta ekranu startowego

8.2 Ekran logowania i rejestracji

0

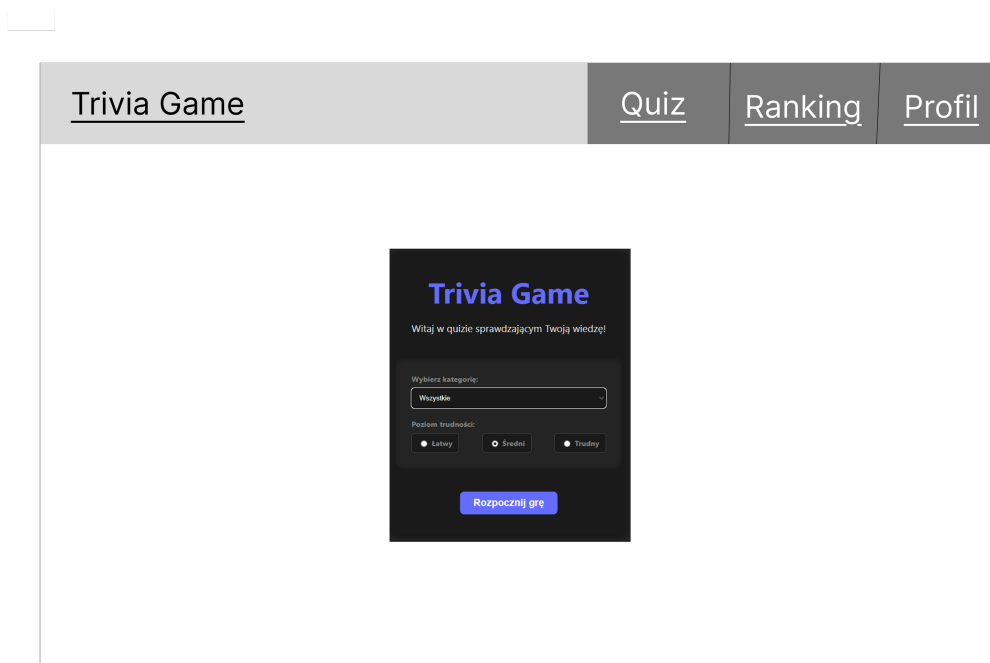


Rysunek 2: Makieta ekranu logowania



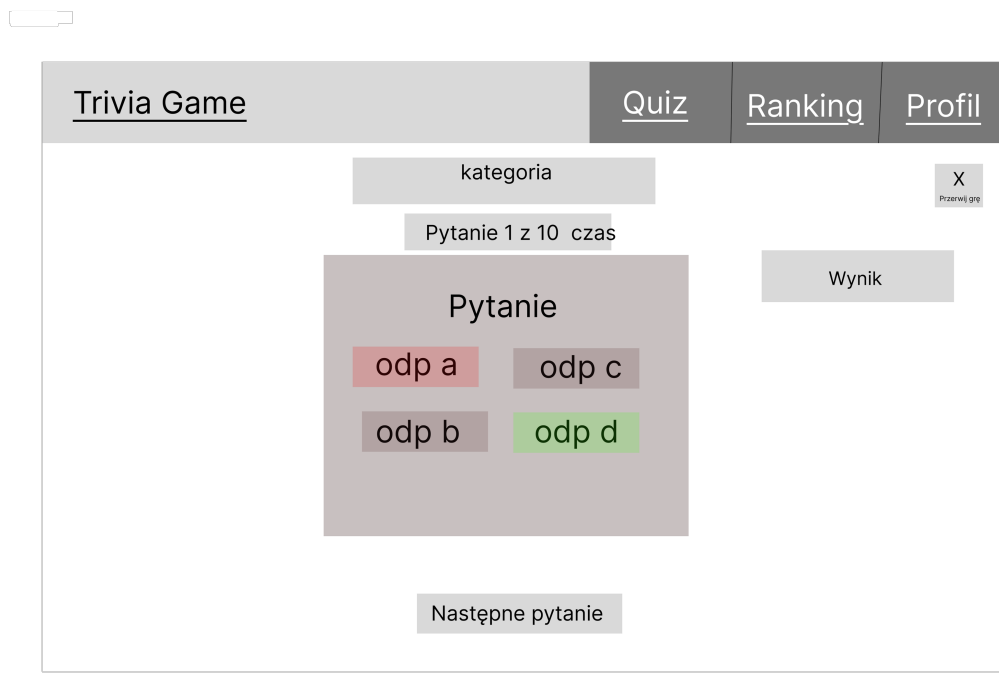
Rysunek 3: Makieta ekranu rejestracji

8.3 Ekran ustawień quizu



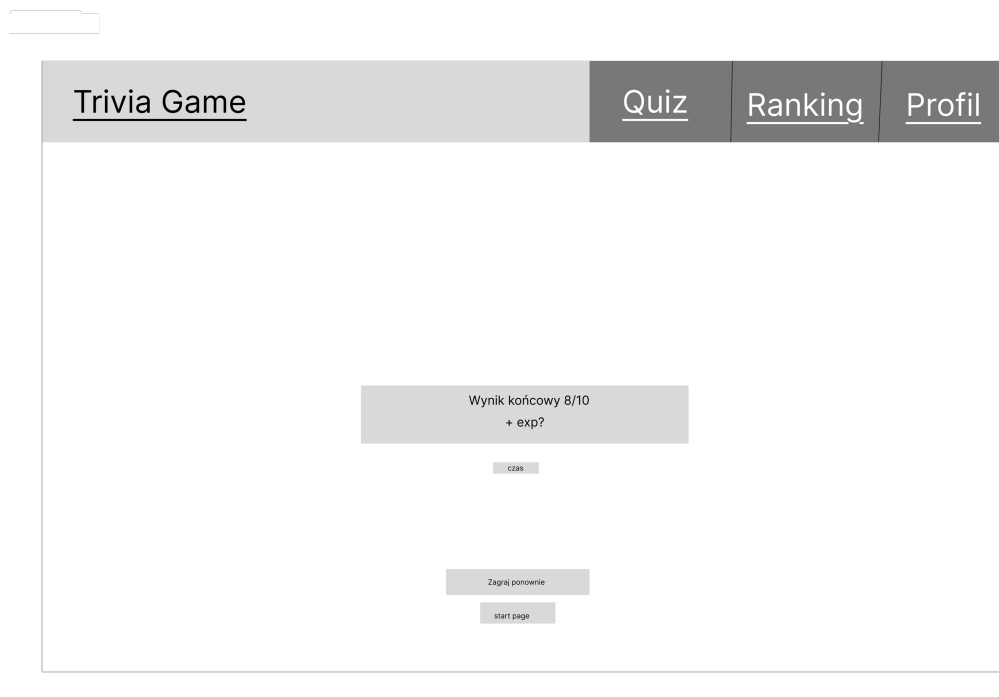
Rysunek 4: Makieta ekranu ustawień gry

8.4 Ekran quizu



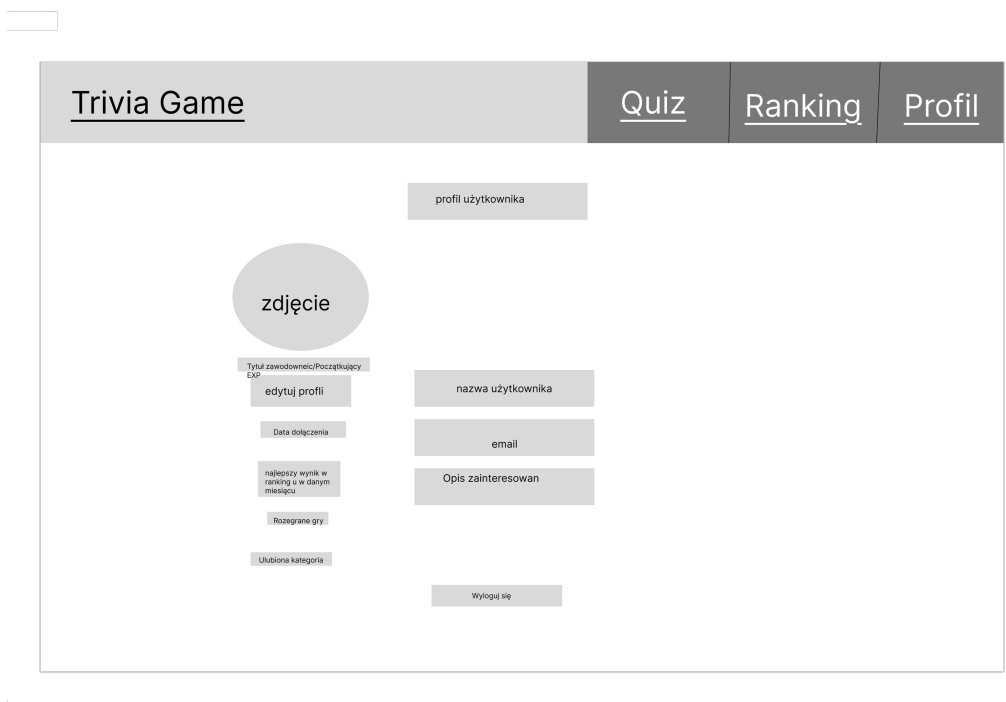
Rysunek 5: Makieta ekranu gry

8.5 Ekran wyniku



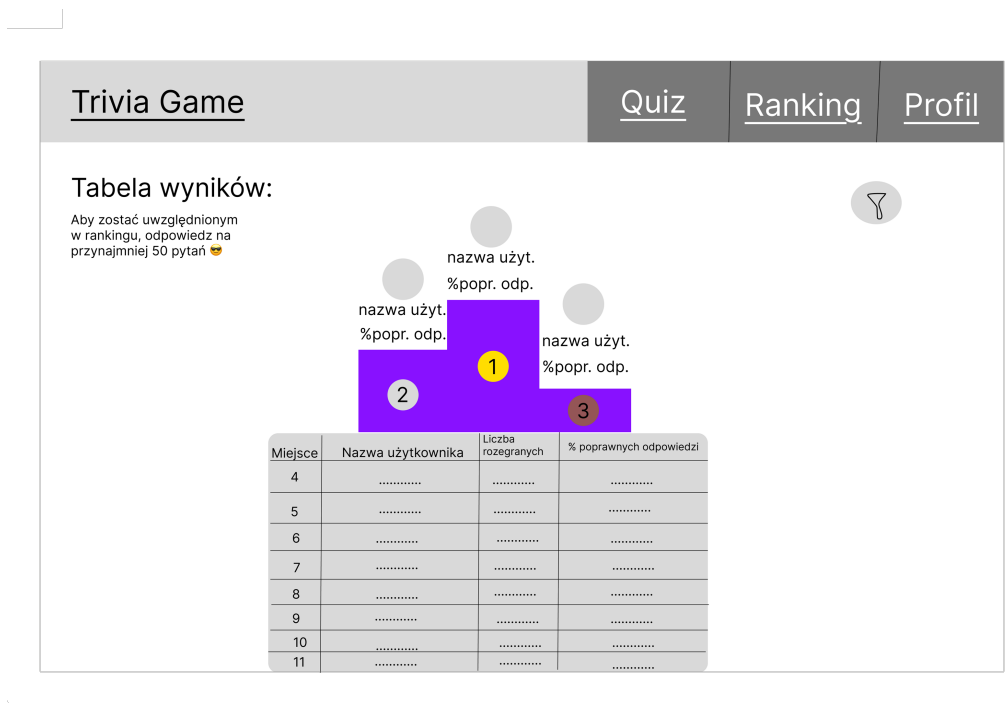
Rysunek 6: Makieta ekranu wyniku

8.6 Profil użytkownika



Rysunek 7: Makieta ekranu profilu użytkownika

8.7 Ranking



Rysunek 8: Makieta ekranu ranking u

9 Specyfikacja techniczna

9.1 Architektura systemu

System zostanie wykonany w architekturze trójwarstwowej:

- frontend odpowiedzialny za interfejs użytkownika,
- backend odpowiedzialny za logikę biznesową i komunikację z API,
- baza danych odpowiedzialna za przechowywanie wyników oraz danych użytkowników.

9.2 Frontend

Frontend aplikacji zostanie wykonany z wykorzystaniem React.js. Główne widoki będą obejmowały ekran startowy, ekran konfiguracji quizu, ekran pytań oraz ekran końcowego wyniku. Interfejs będzie komunikował się z backendem poprzez REST API.

9.3 Backend

Backend zostanie wykonany z wykorzystaniem Bun.js i frameworka Hono. Backend będzie odpowiedzialny za pobieranie danych z Open Trivia Database API, obsługę logiki quizu, walidację odpowiedzi oraz zapis wyników do bazy danych.

9.4 Integracja z Open Trivia Database API

Aplikacja będzie wykorzystywać Open Trivia Database API do pobierania pytań quizowych. Backend będzie wysyłał zapytania do API na podstawie wybranej kategorii i poziomu trudności, a następnie przekazywał dane do frontendu.

9.5 Repozytorium projektu

Kod projektu będzie przechowywany w repozytorium GitHub. Repozytorium będzie zawierało kod źródłowy, dokumentację techniczną, diagramy UML oraz tablicę projektową Kanban.

10 Projekt bazy danych

10.1 Założenia bazy danych

Baza danych aplikacji została zaprojektowana z wykorzystaniem relacyjnego modelu danych. Do przechowywania danych wykorzystana zostanie baza PostgreSQL wraz z narzędziem Prisma ORM.

Głównym celem bazy danych jest przechowywanie informacji o użytkownikach, sesjach quizowych oraz wynikach uzyskanych podczas rozwiązywania quizów. Dane te pozwolą użytkownikowi na przeglądanie historii rozegranych quizów, analizy swoich wyników oraz porównywania ich z innymi użytkownikami w rankingu.

Model bazy danych został zaprojektowany w sposób umożliwiający łatwe rozszerzanie aplikacji o nowe funkcjonalności, takie jak dodatkowe tryby gry, osiągnięcia użytkowników czy zapis pytań quizowych w bazie danych.

10.2 Model użytkownika

Encja User odpowiada za przechowywanie podstawowych informacji o użytkowniku aplikacji.

Przechowywane dane obejmują:

- identyfikator użytkownika,
- nazwę użytkownika,
- adres e-mail,
- hasło w postaci zaszyfrowanej,
- datę utworzenia konta,
- datę ostatniej modyfikacji danych.

Każdy użytkownik może posiadać wiele sesji quizowych oraz wiele zapisanych wyników.

10.3 Model quizu i wyników

Encja QuizSession odpowiada za przechowywanie informacji o pojedynczej sesji quizowej użytkownika.

Dla każdej sesji zapisywane są między innymi:

- identyfikator sesji,
- identyfikator użytkownika,
- status sesji,
- liczba zdobytych punktów,
- liczba pytań,
- wybrana kategoria,
- poziom trudności,
- data rozpoczęcia quizu,
- data zakończenia quizu.

Encja QuizResult odpowiada za przechowywanie szczegółowych informacji o odpowiedziach udzielonych przez użytkownika.

Dla każdego pytania zapisywane są:

- treść pytania,
- odpowiedź użytkownika,
- poprawna odpowiedź,
- informacja, czy odpowiedź była poprawna,
- data zapisania wyniku.

10.4 Relacje między encjami

Pomiędzy encjami występują relacje typu jeden do wielu.

Jeden użytkownik może posiadać wiele sesji quizowych. Każda sesja quizowa należy jednak wyłącznie do jednego użytkownika.

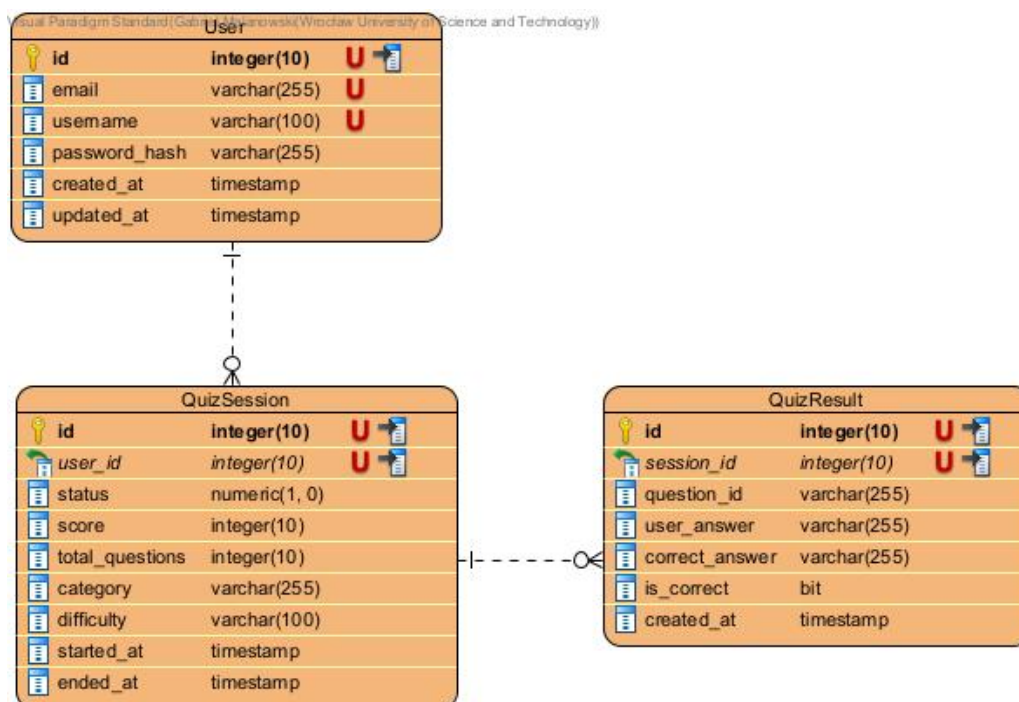
Jedna sesja quizowa może zawierać wiele wyników dotyczących poszczególnych pytań. Każdy wynik jest przypisany wyłącznie do jednej sesji quizowej.

Takie podejście pozwala zachować spójność danych oraz łatwo analizować historię aktywności użytkownika.

10.5 Schemat bazy danych

Schemat bazy danych przedstawia trzy główne encje: User, QuizSession oraz QuizResult. Relacje między encjami zostały zaprojektowane zgodnie z zasadami normalizacji danych.

Każdy użytkownik może posiadać wiele sesji quizowych, natomiast każda sesja może posiadać wiele wyników. Dzięki temu możliwe jest przechowywanie pełnej historii rozgrywek użytkownika.

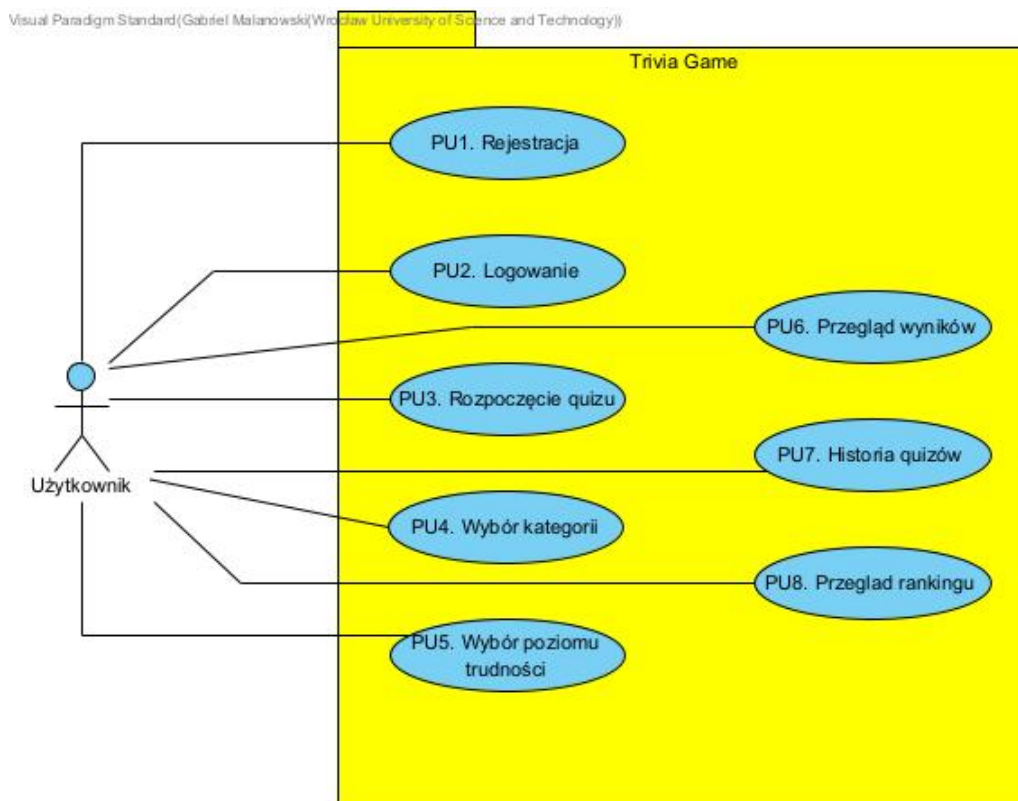


Rysunek 9: Diagram ERD

11 Diagramy UML

11.1 Diagram przypadków użycia

Diagram przypadków użycia przedstawia główne funkcjonalności aplikacji z perspektywy użytkownika. Użytkownik może między innymi zarejestrować się, zalogować, rozpocząć quiz, wybrać kategorię oraz poziom trudności, rozwiązywać quiz, przeglądać wyniki, historię rozgrywek oraz ranking użytkowników.

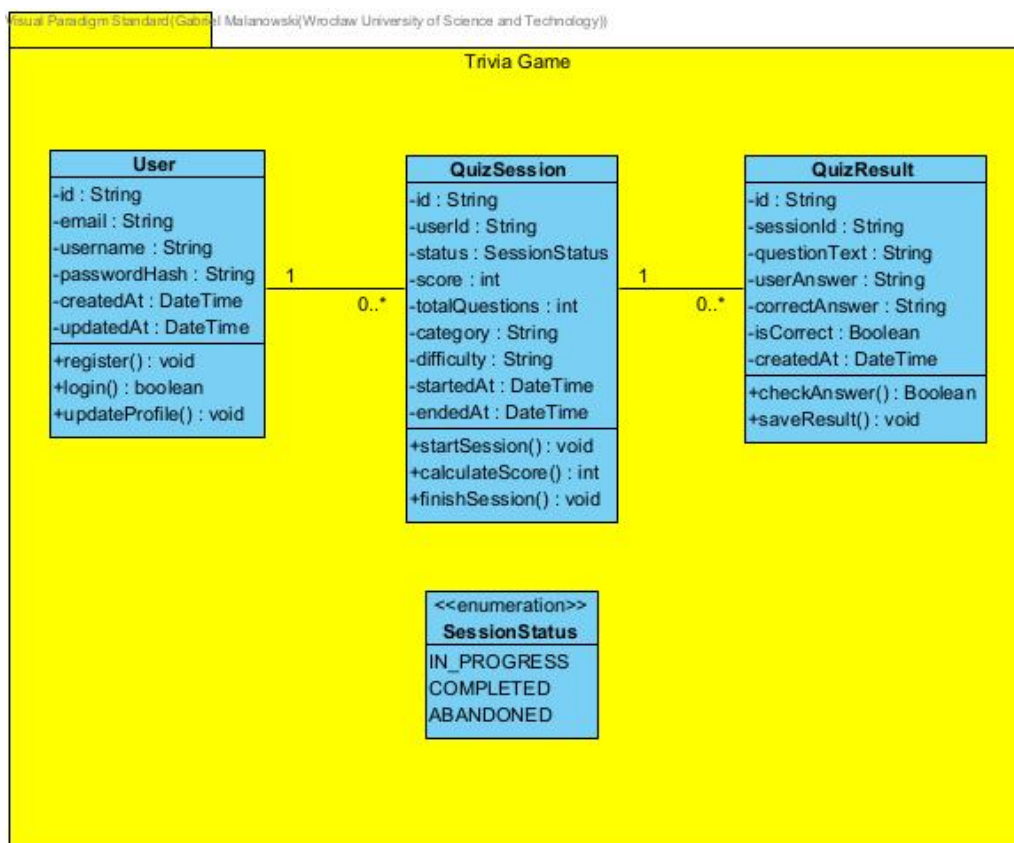


Rysunek 10: Diagram przypadków użycia

11.2 Diagram klas

Diagram klas przedstawia główne encje wykorzystywane w systemie oraz relacje pomiędzy nimi. W projekcie wyróżniono trzy podstawowe klasy: User, QuizSession oraz QuizResult.

Klasa User przechowuje informacje o użytkowniku. Klasa QuizSession odpowiada za informacje o przebiegu quizu, natomiast QuizResult przechowuje szczegółowe dane dotyczące odpowiedzi udzielonych przez użytkownika.

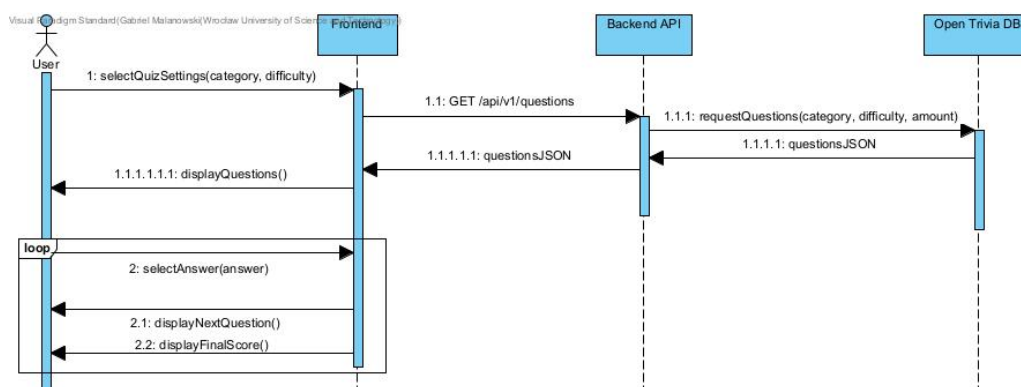


Rysunek 11: Diagram klas

11.3 Diagram sekwencji

Diagram sekwencji przedstawia komunikację pomiędzy użytkownikiem, frontendem, backendem oraz zewnętrznym API Open Trivia Database.

Proces rozpoczyna się od wyboru parametrów quizu przez użytkownika. Następnie frontend wysyła zapytanie do backendu, który pobiera pytania z zewnętrznego API. Po otrzymaniu odpowiedzi backend zwraca dane do frontendu w formacie JSON, a użytkownik może rozpocząć rozwiązywanie quizu.

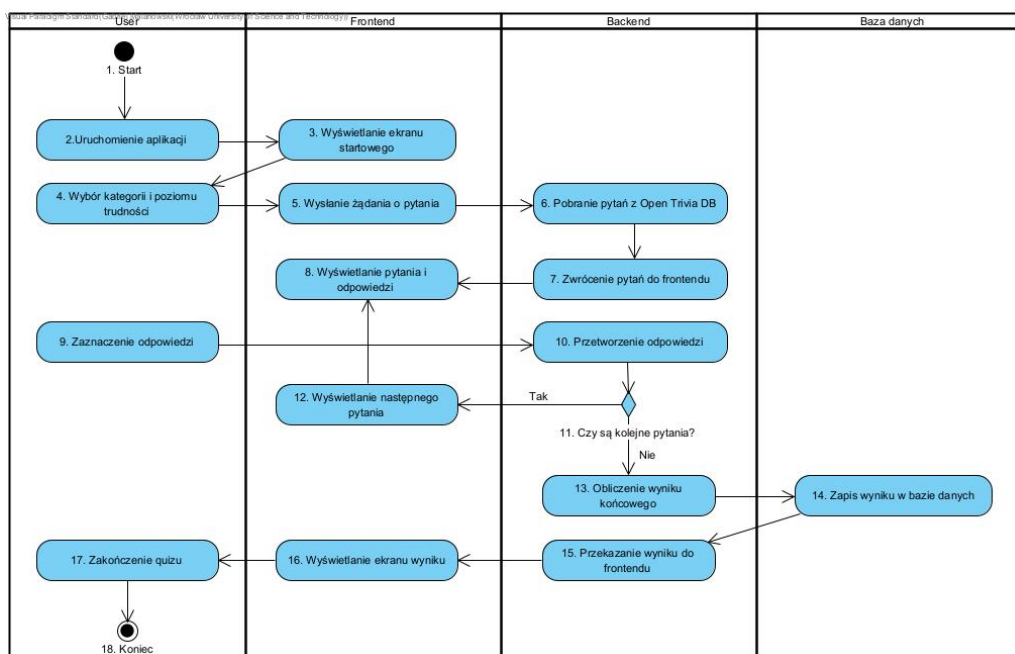


Rysunek 12: Diagram sekwencji

11.4 Diagram aktywności

Diagram aktywności przedstawia przebieg procesu rozwiązywania quizu.

Proces rozpoczyna się od uruchomienia aplikacji i wyboru parametrów quizu. Następnie pobierane są pytania, użytkownik udziela odpowiedzi, a system sprawdza, czy pozostały jeszcze kolejne pytania. Po zakończeniu quizu następuje obliczenie wyniku i wyświetlenie podsumowania użytkownikowi.



Rysunek 13: Diagram aktywności

12 Organizacja pracy zespołu

12.1 Metodyka pracy

Projekt będzie realizowany zgodnie z frameworkiem Scrum. Zespół będzie pracował iteracyjnie, a postępy będą omawiane podczas regularnych spotkań.

12.2 Narzędzia komunikacji

Do komunikacji wykorzystywany będzie Messenger. Dokumentacja, zadania oraz postępy będą przechowywane w repozytorium GitHub.

12.3 Zarządzanie zadaniami

Do zarządzania zadaniami wykorzystywana będzie tablica Kanban w GitHub Projects z kolumnami:

- Backlog,
- Todo,
- In Progress,
- Done.

12.4 Workflow Git i Pull Requestów

Każda nowa funkcjonalność będzie realizowana w osobnej gałęzi repozytorium. Zmiany będą integrowane z główną gałęzią projektu za pomocą Pull Requestów po wcześniejszym code review.

13 Analiza ryzyka

13.1 Ryzyka techniczne

- problemy z integracją Open Trivia Database API,
- błędy komunikacji frontend-backend,
- problemy z konfiguracją bazy danych,
- trudności z integracją wszystkich komponentów systemu.

13.2 Ryzyka organizacyjne

- nierównomierny podział obowiązków,
- problemy z komunikacją w zespole,
- brak terminowego wykonywania zadań,
- konflikty dotyczące sposobu implementacji.

13.3 Ryzyko opóźnień

- opóźnienia w implementacji backendu,
- opóźnienia w przygotowaniu frontendu,
- konieczność poprawiania wcześniej wykonanych funkcjonalności,
- problemy wynikające z sesji egzaminacyjnej i innych obowiązków członków zespołu.

13.4 Sposoby monitorowania ryzyka

- regularne spotkania zespołu,
- aktualizacja tablicy Kanban,
- monitorowanie postępów w repozytorium GitHub,
- kontrola realizacji harmonogramu.

13.5 Plan awaryjny

W przypadku problemów technicznych lub organizacyjnych zespół będzie ograniczał zakres projektu do funkcjonalności MVP. Funkcje dodatkowe, takie jak ranking użytkowników czy rozbudowane profile, będą mogły zostać przesunięte na późniejsze etapy realizacji projektu.

14 Wykorzystanie narzędzi AI

14.1 Zakres wykorzystania AI

14.2 Narzędzia AI użyte w projekcie

14.3 Przykłady zastosowania

14.4 Deklaracja wykorzystania AI

15 Podsumowanie

15.1 Osiągnięte cele

15.2 Aktualny stan projektu

15.3 Możliwe kierunki rozwoju